

FixedIncomeLib - User Guide

How to load data, build models, value products, and generate risk reports

February 15, 2026

This guide focuses on: (i) data ingestion, (ii) yield curve construction, (iii) SABR surface models, (iv) product valuation, and (v) first-order risk & reporting.

Contents

1	Install & quick import	2
2	Core workflow	4
3	Glossary / key map	5
4	Data layer	6
5	Conventions & indices	7
6	Building a yield curve	8
7	Products	10
8	Valuation engines and registry	11
9	Building SABR surfaces	12
10	Valuation examples	14
11	First-order risk and reporting	15
12	How to extend	16
13	Notebooks as executable documentation	18
14	Troubleshooting	19

1 Install & quick import

1.1 Prerequisites

- Python **3.10+** (tested on **3.11**)
- OS: macOS / Linux (Windows supported via WSL2)

1.2 Install

From the repository root:

```
python -m pip install -U pip setuptools wheel
pip install -r requirements.txt
pip install -e .
```

1.3 Quick import check

```
from fixedincomelib.date import Date
print(Date("2025-09-24"))
```

1.4 15-minute quickstart

The quickest end-to-end workflow is:

1. create a minimal SOFR market quote table,
2. build a SOFR curve (SOFR futures + SOFR OIS),
3. price a simple overnight swap,
4. generate a small risk table in parameter and quote space.

1.4.1 Copy-paste script

```
import pandas as pd

from fixedincomelib.yield_curve import YieldCurve
from fixedincomelib.data.yc_market_loader import build_yc_data_collection
from fixedincomelib.product import ProductOvernightSwap
from fixedincomelib.valuation import ValuationEngineRegistry
from fixedincomelib.utilities.risk_reporting import (
    createValueReport,
    risk_vectors_to_df
)

# 1) Market quotes (minimal illustrative set)
# - Futures: quoted as PRICE (e.g., 95.70)
# - OIS swaps: quoted as RATE (e.g., 0.0280)
MARKET_DF = pd.DataFrame(
    [
        ["RFR FUTURE", "SOFR-FUTURE-3M", "2025-09-24 x 2025-12-24", 95.70],
        ["RFR FUTURE", "SOFR-FUTURE-3M", "2025-12-24 x 2026-03-24", 95.80],
```

```

        ["RFR FUTURE", "SOFR-FUTURE-3M", "2026-03-24 x 2026-06-24", 95.85],
        ["RFR SWAP", "USD-SOFR-OIS", "1Y", 0.0275],
        ["RFR SWAP", "USD-SOFR-OIS", "2Y", 0.0280],
        ["RFR SWAP", "USD-SOFR-OIS", "5Y", 0.0290],
    ],
    columns=["INSTRUMENT", "DATA CONVENTION", "TERM", "QUOTE"]
)

# 2) Convert market dataframe to DataCollection
data_objs, dc = build_yc_data_collection(MARKET_DF)

# 3) Curve build methods
build_methods = [{
    "TARGET": "SOFR-1B",
    "REFERENCE": None,
    "INSTRUMENTS": ["SOFR-FUTURE-3M", "USD-SOFR-OIS"],
    "INTERPOLATION METHOD": "PIECEWISE_CONSTANT",
}]

# 4) Build the curve
valueDate = "2025-09-24"
yc = YieldCurve(valueDate, dc, build_methods)

# 5) Product: simple overnight swap
swap = ProductOvernightSwap(
    effectiveDate="2025-09-24",
    maturityDate="2027-09-24",
    frequency="6M",
    overnightIndex="SOFR-1B",
    spread=0.0,
    fixedRate=0.03,
    notional=10_000_000,
    position="LONG",
    holConv="SOFR",
    bizConv="F",
    accrualBasis="ACT/360",
    rule="BACKWARD",
    endOfMonth=False,
)

# 6) PV + risk (engine)
vp = {"FUNDING INDEX": "SOFR-1B"}
ve = ValuationEngineRegistry().new_valuation_engine(yc, vp, swap)

ve.calculateValue()
ccy, pv = ve.value_
print(f"PV ({ccy}):", float(pv))

ve.calculateFirstOrderRisk()
g = ve.firstOrderRisk_
print("firstOrderRisk shape:", getattr(g, "shape", (len(g),)))

# 7) Report helper (parameter + quote space)
report_p = createValueReport(vp, yc, swap, request="all", space="pv")

```

```

report_q = createValueReport(vp, yc, swap, request="all", space="quote")

print("PV (report):", float(report_p["pv"]))
print("param_risk length:", len(report_p["param_risk"]))
print("quote_risk length:", len(report_q["quote_risk"]))

# 8) Small risk table (top rows)
df_risk = risk_vectors_to_df(yc, report_q, yc_index="SOFR-1B")
print(df_risk.head(10))

```

1.4.2 Expected output format (with units)

Exact numbers depend on the quotes, curve settings, and your swap rate. The output should look like:

```

PV (currency units): 12345.67
firstOrderRisk length: N
PV (report): 12345.67
param_risk length: N
quote_risk length: N

   block  node_id  ...  dPV_dtheta  hedge_weight  quote_risk
0    YC    ...    ...    ...    ...    ...
1    YC    ...    ...    ...    ...    ...
...

```

Interpretation:

- **PV** is reported in **currency units** (consistent with the product notional).
- **dPV_dtheta** is sensitivity w.r.t. internal model parameters (pillar/curve state).
- **hedge_weight** are the mapped hedge weights solving $J^\top w = g$.
- **quote_risk** is the same risk expressed in **quote space** using the model's per node quote sensitivity scaling.

2 Core workflow

1. Create a market quote table (often a `pandas.DataFrame`).
2. Convert market quotes to a `DataCollection` (via `build_yc_data_collection` for yield curve quotes, or `Data2D.createDataObject` for SABR surfaces).
3. Specify a `buildMethodCollection` (a list of dictionaries).
4. Construct a model (`YieldCurve` or `SabrModel`).
5. Build products (`Product...` classes), then value and compute risk via the valuation engine registry.
6. Optionally create a reporting dataframe via `utilities.risk_reporting`.

3 Glossary / key map

3.1 Data objects and keying

Object / key	Meaning
Data1D	1D quotes: axis $\rightarrow$ value (e.g. tenor $\rightarrow$ par rate).
Data2D	2D grid: axis1 $\times$ axis2 $\rightarrow$ value (e.g. expiry $\times$ tenor $\rightarrow$ SABR param).
DataCollection	Container mapping (data_type, data_convention) $\rightarrow$ Data1D/Data2D.
data_type	What the values represent (e.g. "QUOTE" series for YC; "NORMALVOL/BETA/NU/RHO" for SABR).
data_convention	How to interpret the axis/value (YC: convention key like "USD-SOFR-OIS"; SABR: index key like "SOFR-1B").

3.2 Naming conventions (indices vs data conventions)

Name	Meaning
"SOFR-1B"	Index key: used to identify curve components and forwarding/discounting.
"SOFR-FUTURE-3M"	Data convention key: used by the market loader to create calibration instruments.
"USD-SOFR-OIS"	Data convention key: used to interpret swap quotes as OIS instruments.

3.3 Build-method dictionary keys (most common)

YieldCurve build-method keys.

Key	Meaning
"TARGET"	Index key for the curve component (e.g. "SOFR-1B").
"REFERENCE"	Reserved for multi-curve / references (often <code>None</code> in notebooks).
"INSTRUMENTS"	List of data convention keys to include (e.g. futures + OIS).
"INTERPOLATION METHOD"	Curve interpolation rule (e.g. "PIECEWISE_CONSTANT").

SabrModel build-method keys.

Key	Meaning
"TARGET"	Index key (e.g. "SOFR-1B").
"VALUES"	One of NORMALVOL/BETA/NU/RHO.
"INTERPOLATION"	Surface interpolation rule (risk currently expects "LINEAR").
"SHIFT"	Shift stored on the component (for shifted lognormal variants).
"VOL_DECAY_SPEED"	Time decay speed used by decay analytics.
"PRODUCT"	Optional suffix to support product-specific surfaces.

3.4 Quote conventions and units

Instrument quote	Typical unit
RFR futures price	Price (e.g. 95.70), typically implies an annualized rate via $100 - \text{price}$.
OIS swap par rate	Rate in decimal (e.g. $0.0280 = 2.80\%$).
SABR parameters	Dimension depends on parameter; NORMALVOL is in normal-vol units; RHO is dimensionless correlation.

4 Data layer

4.1 Data1D, Data2D, and DataCollection

`fixedincomelib.data` defines:

- **Data1D**: 1D quotes (`axis + values`), tagged by `data_type` and `data_convention`.
- **Data2D**: 2D grids (`axis1, axis2, values`), tagged similarly.
- **DataCollection**: a container mapping (`data_type, data_convention`) $\rightarrow$ data object.

In the yield curve path, `Data1D.data_convention` is typically the *instrument convention key* such as "SOFR-FUTURE-3M" or "USD-SOFR-OIS" (see `fixedincomelib/conventions/data_convention_registry`). In the SABR path, `Data2D.data_convention` is the *index key* such as "SOFR-1B".

4.2 Loading yield curve market quotes: `build_yc_data_collection`

The notebooks use `build_yc_data_collection(MARKET_DF)` from `fixedincomelib.data.yc_market_loader`. A valid input frame matches:

Column	Example	Meaning
INSTRUMENT	"RFR FUTURE"	instrument type label (used by loader)
DATA CONVENTION	"SOFR-FUTURE-3M"	convention key (maps to <code>DataConvention</code>)
TERM	"2025-09-24 x 2025-12-24" or "5Y"	axis encoding
QUOTE	95.70 or 0.0285	market quote

Example (from `tests/risk_calculation_check.ipynb`):

```
import pandas as pd
from fixedincomelib.data import build_yc_data_collection

MARKET_DF = pd.DataFrame(
    [
        ["RFR FUTURE", "SOFR-FUTURE-3M", "2025-09-24 x 2025-12-24", 95.70],
        ["RFR FUTURE", "SOFR-FUTURE-3M", "2025-12-24 x 2026-03-24", 95.80],
        # ...
        ["RFR SWAP", "USD-SOFR-OIS", "2Y", 0.0280],
        ["RFR SWAP", "USD-SOFR-OIS", "5Y", 0.0290],
    ],
    columns=["INSTRUMENT", "DATA CONVENTION", "TERM", "QUOTE"]
```

```
)
data_objs, dc = build_yc_data_collection(MARKET_DF)
```

How the loader encodes axes. For "SOFR-FUTURE-3M", the TERM string uses the format "YYYY-MM-DD x YYYY-MM-DD", and the loader stores the axis entry as a tuple (`start_iso`, `end_iso`). For swap conventions such as "USD-SOFR-OIS", the loader stores TERM as a tenor string like "2Y".

5 Conventions & indices

5.1 Data conventions

`fixedincomelib.conventions` defines `DataConvention` classes and a registry in `data_convention_registry.json`. In this repository, two conventions are provided out of the box:

- "SOFR-FUTURE-3M" (`kind="RFR FUTURE"`)
- "USD-SOFR-OIS" (`kind="RFR SWAP"`)

These conventions drive instrument construction through the product-builder registry.

5.2 Index registry

`fixedincomelib.market` contains an index registry (see `fixedincomelib/market/index_registry.json`) and index classes in `fixedincomelib/market/indices.py`. Index objects encapsulate day count, calendars, and business-day conventions. These are used consistently across:

- schedule generation (cashflows and swaps),
- accrual computation,
- forward-rate and discount-factor calculations.

6 Building a yield curve

6.1 Build methods

A yield curve is constructed with:

```
from fixedincomelib.yield_curve import YieldCurve
yc = YieldCurve(valueDate, dc, build_methods)
```

A yield curve build method is a dictionary. In this repo, the notebooks use:

- "TARGET": the index key for the curve component (e.g. "SOFR-1B")
- "REFERENCE": currently set to `None` in notebooks
- "INSTRUMENTS": list of **data convention keys** to include (e.g. ["SOFR-FUTURE-3M", "USD-SOFR-OIS"])
- "INTERPOLATION METHOD": e.g. "PIECEWISE CONSTANT"

Example:

```
build_methods = [{
    "TARGET": "SOFR-1B",
    "REFERENCE": None,
    "INSTRUMENTS": ["SOFR-FUTURE-3M", "USD-SOFR-OIS"],
    "INTERPOLATION METHOD": "PIECEWISE_CONSTANT",
}]
yc = YieldCurve("2025-09-24", dc, build_methods)
```

6.2 Calibration instruments

Market quotes alone are not sufficient to calibrate a yield curve or a volatility surface. Each quote must first be interpreted as a *tradable instrument* with a precise payoff, schedule, and valuation logic.

For example:

- a SOFR future quote represents the price of a futures contract with a specific accrual window,
- an OIS swap quote represents a fixed-vs-floating stream of cashflows,
- a caplet quote represents an option on a compounded overnight rate.

Calibration in this library is therefore formulated as:

choose model parameters such that the model PV of each calibration instrument matches its market quote.

This design ensures that:

- the same instrument definitions are used for calibration, valuation, and risk,
- pricing logic is not duplicated or approximated during calibration,
- Jacobians and first-order risk are computed in a consistent and reusable way.

6.3 Calibration basket construction

To implement this idea, the library converts market data into an explicit set of calibration instruments. Internally, `YieldCurveModelComponent.calibrate()` constructs a `CalibrationBasket` from the `DataCollection` using:

```
build_yc_calibration_basket_from_dc(value_date, data_collection, build_method).
```

Each entry in the basket is a `CalibItem` containing:

- the instrument object used for pricing,
- the associated market quote,
- the data type and data convention that produced it,
- the axis identifier (e.g. tenor or date tuple).

The calibration algorithm repeatedly values these instruments under the current model parameters and adjusts the model state until pricing errors are minimized.

6.4 Discount factors and forwards

`YieldCurve` exposes:

- `discountFactor(index, to_date)`
- `forward(index, effectiveDate, termOrTerminationDate=)`

IBOR vs OIS forwarding. `forward()` checks whether the component index is overnight.

- Overnight: must pass a term (e.g. "3M") or an explicit end date; forwarded via `forwardOvernightIndex`.
- IBOR: if an explicit end date is passed and it differs from the index tenor end date, the method raises an error indicating a non-vanilla coupon (stub handling is not enabled in this method).

Example:

```
df_2y = yc.discountFactor("SOFR-1B", "2027-09-24")
fwd_3m = yc.forward("SOFR-1B", "2026-07-01", "3M")
```

7 Products

Products live in `fixedincomelib.product`. They are grouped into:

- `linear_products.py`: cashflows, futures, swaps, portfolios
- `non_linear_products.py`: caplets/capfloors and swaptions (IBOR and overnight)

7.1 Linear products

- `ProductBulletCashflow`
- `ProductIborCashflow`
- `ProductOvernightIndexCashflow`
- `ProductFuture`
- `ProductRfrFuture`
- `ProductIborSwap` and `ProductOvernightSwap`
- `ProductPortfolio` (container/aggregation)

7.2 Non-linear products

- `ProductIborCapFloorlet`, `ProductOvernightCapFloorlet`
- `ProductIborCapFloor`, `ProductOvernightCapFloor`
- `ProductIborSwaption`, `ProductOvernightSwaption`

8 Valuation engines and registry

Pricing is routed through `ValuationEngineRegistry`:

```
from fixedincomelib.valuation import ValuationEngineRegistry
ve = ValuationEngineRegistry().new_valuation_engine(model,
    valuation_parameters, product)
```

8.1 yield curve engines

`fixedincomelib/yield_curve/valuation_engine_yc.py` registers engines for:

- bullet cashflow, IBOR cashflow, overnight cashflow
- futures and RFR futures
- swaps via a unified `ValuationEngineInterestRateStream`
- portfolios via `ValuationEngineProductPortfolio`

8.2 SABR engines

`fixedincomelib/sabr/valuation_engine_sabr.py` registers engines for:

- IBOR cap/floorlets and caps/floors
- overnight cap/floorlets and caps/floors
- IBOR swaptions and overnight swaptions

SABR valuation uses `fixedincomelib.analytics.SABRCalculator`. For IBOR products, the engine forces standard Hagan SABR (and warns if a different method is requested). For overnight products, the engine supports additional SABR risk modes via `valuation_parameters["SABR_METHOD"]` and (optionally) a correlation surface input `valuation_parameters["CORR_DF"]`.

9 Building SABR surfaces

9.1 Creating SABR surfaces as Data2D

SABR parameters are stored as separate Data2D objects per parameter: NORMALVOL, BETA, NU, RHO. Each parameter is pivoted into a grid indexed by:

- AXIS1: expiry (year-fraction)
- AXIS2: tenor (year-fraction)

From tests/risk_check_sabr_plain_hagan.ipynb:

```
from fixedincomelib.data import Data2D, DataCollection

PARAMS = ["NORMALVOL", "BETA", "NU", "RHO"]
sabr_objs = []
for idx_name, sub in sabr_sofr.groupby("INDEX"):
    for param in PARAMS:
        pivot = (
            sub.pivot(index="AXIS1", columns="AXIS2", values=param)
            .sort_index(axis=0).sort_index(axis=1)
        )
        sabr_objs.append(
            Data2D.createDataObject(
                data_type=param, # stored as param.lower()
                data_convention=idx_name, # e.g. "SOFR-1B"
                df=pivot
            )
        )
sabr_dc = DataCollection(sabr_objs)
```

9.2 SABR build methods

A SABR build method entry is:

- "TARGET": index key (e.g. "SOFR-1B")
- "VALUES": one of NORMALVOL/BETA/NU/RHO
- "INTERPOLATION": currently "LINEAR" (enforced for risk)
- "SHIFT": stored in the component
- "VOL_DECAY_SPEED": stored in the component (used by time-decay analytics)
- Optional "PRODUCT": adds a suffix to the component key for product-specific surfaces

```
sabr_build_methods = [
    {
        "TARGET": "SOFR-1B",
        "VALUES": param,
        "INTERPOLATION": "LINEAR",
        "SHIFT": 0.0,
        "VOL_DECAY_SPEED": 0.2
    }
    for param in ("NORMALVOL", "BETA", "NU", "RHO")
]
```

9.3 Constructing a SabrModel

The standard path (used in the notebooks) is to first build a `YieldCurve` and then attach SABR to it:

```
from fixedincomelib.sabr import SabrModel

sabr_model = SabrModel.from_curve(
    valueDate="2025-09-24",
    dataCollection=sabr_dc,
    buildMethodCollection=sabr_build_methods,
    ycModel=yc
)
```

A convenience constructor also exists to build the curve from a market dataframe:

```
sabr_model = SabrModel.from_data(
    valueDate="2025-09-24",
    dataCollection=sabr_dc,
    buildMethodCollection=sabr_build_methods,
    yc_market_df=MARKET_DF,
    ycBuildMethods=build_methods
)
```

10 Valuation examples

10.1 Example: yield curve valuation

```
from fixedincomelib.product import ProductOvernightSwap
from fixedincomelib.valuation import ValuationEngineRegistry

swap = ProductOvernightSwap(
    effectiveDate="2025-09-24",
    maturityDate="2027-09-24",
    frequency="6M",
    overnightIndex="SOFR-1B",
    spread=0.0,
    fixedRate=0.03,
    notional=10_000_000,
    position="LONG",
    holConv="SOFR",
    bizConv="F",
    accrualBasis="ACT/360",
    rule="BACKWARD",
    endOfMonth=False,
)
vp = {"FUNDING_INDEX": "SOFR-1B"}
ve = ValuationEngineRegistry().new_valuation_engine(yc, vp, swap)
ve.calculateValue()
print("PV (currency units):", float(ve.value))
```

10.2 Example: SABR valuation (overnight caplet)

From tests/risk_check_sabr_bottom_up_check.ipynb:

```
from fixedincomelib.product import ProductOvernightCapFloorlet
from fixedincomelib.valuation import ValuationEngineRegistry

caplet = ProductOvernightCapFloorlet(
    effectiveDate="2026-07-01",
    termOrEnd="3M",
    index="SOFR-1B",
    compounding="COMPOUND",
    optionType="CAP",
    strike=0.018,
    notional=1_000_000,
    longOrShort="LONG",
)

vp = {"SABR_METHOD": "bottom-up", "CORR_DF": df_corr}
ve = ValuationEngineRegistry().new_valuation_engine(sabr_model, vp, caplet)

ve.calculateValue()
print("PV (currency units):", float(ve.value))

ve.calculateFirstOrderRisk()
```

```
print("firstOrderRisk vector shape:", len(ve.firstOrderRisk))
```

11 First-order risk and reporting

11.1 Model Jacobian and quote sensitivity

Both `YieldCurve` and `SabrModel` implement:

- `jacobian()`: builds a square matrix by valuing each calibration instrument and collecting the model gradient.
- `calibration_quote_sensitivity()`: a per-node scaling used when reporting risk in quote space.

11.2 Reporting helper: `createValueReport`

`fixedincomelib.utilities.risk_reporting.createValueReport` is the main reporting helper used in notebooks. It returns:

- PV
- `g_theta`: the first-order risk vector from the valuation engine
- `param_risk`: the mapped hedge weights from solving $J^T w = g$
- optionally `quote_risk` if `space="quote"`

```
from fixedincomelib.utilities.risk_reporting import createValueReport

report = createValueReport(vp, sabr_model, caplet, request="all", space="
    pv")
print("PV (currency units):", float(report["pv"]))
print("param_risk length:", report["param_risk"].shape)

report_q = createValueReport(vp, sabr_model, caplet, request="all", space=
    "quote")
print("quote_risk length:", report_q["quote_risk"].shape)
```

11.3 Tabular risk: `risk_vectors_to_df`

`risk_vectors_to_df(model, report)` converts report vectors into a tidy dataframe with:

- block identification (YC vs SABR),
- node metadata (expiry/tenor for SABR; node id for YC),
- $\partial PV / \partial \theta$ entries,
- hedge weights in parameter space,
- optional quote-space risk.

```

from fixedincomelib.utilities.risk_reporting import risk_vectors_to_df

df = risk_vectors_to_df(sabr_model, report_q, yc_index="SOFR-1B")
print(df.head())

```

12 How to extend

12.1 Add a new instrument convention

Instrument conventions are registered in `fixedincomelib/conventions/data_convention_registry.json`. A new entry typically specifies:

- a unique convention key,
- the kind used by the market loader (e.g. "RFR SWAP", "RFR FUTURE"),
- schedule/day-count/business-day logic required to build the instrument.

A minimal registry entry (illustrative):

```

# data_convention_registry.json (illustrative shape)
{
  "USD-SOFR-OIS": {
    "kind": "RFR SWAP",
    "currency": "USD",
    "index": "SOFR-1B",
    "fixed_frequency": "1Y",
    "float_frequency": "3M",
    "accrualBasis": "ACT/360",
    "bizConv": "F",
    "holConv": "SOFR"
  }
}

```

12.2 Hook the convention into the calibration-basket builder

yield curve calibration baskets are built from the `DataCollection` by mapping each market quote to an instrument object. The typical pattern is:

1. resolve `DataConvention` by key,
2. read the axis encoding (tenor or date tuple),
3. construct the product/instrument object,
4. store it as a `CalibItem` with the quote and metadata.

Skeleton (illustrative pattern):

```
# fixedincomelib/yield_curve/builders/yc_calibration_basket_builder.py (
    illustrative)
def build_item_from_quote(value_date, data_convention, axis, quote):
    if data_convention.kind == "RFR SWAP":
        # build ProductOvernightSwap (or instrument wrapper)
        # return CalibItem(product=swap, quote=quote, ...)
        pass
    elif data_convention.kind == "RFR FUTURE":
        # build ProductRfrFuture (or instrument wrapper)
        pass
    else:
        raise ValueError(f"Unsupported convention kind: {data_convention.
            kind}")
```

12.3 Register a valuation engine hook

Pricing and risk are routed through ValuationEngineRegistry. The usual pattern is:

1. implement ValuationEngine<YourProduct> (value + first-order risk),
2. register it against the product type and model type.

Skeleton:

```
# fixedincomelib/valuation/valuation_engine_registry.py (illustrative
    pattern)
class ValuationEngineMyProduct(...):
    def calculateValue(self):
        ...
    def calculateFirstOrderRisk(self):
        ...

# registry registration (illustrative)
ValuationEngineRegistry().register(
    model_type="YIELD_CURVE",
    product_type="MY_PRODUCT",
    engine_cls=ValuationEngineMyProduct
)
```

13 Notebooks as executable documentation

The following notebooks cover end-to-end usage:

- `tests/risk_calculation_check.ipynb`: YC build + product PV + risk mapping checks.
- `tests/test_valuation.ipynb`: valuation engine usage patterns.
- `tests/test_non_linear_valuation.ipynb`: non-linear products under SABR.
- `tests/risk_check_sabr_plain_hagan.ipynb`: SABR surface build + swaption PV/risk.
- `tests/risk_check_sabr_bottom_up_check.ipynb`: overnight products with bottom-up SABR risk mode.
- `tests/risk_check_sabr_top_down_check.ipynb`: overnight products with top-down SABR risk mode.
- `tests/vol_decay_test.ipynb`: time decay checks for SABR/normalvol.

14 Troubleshooting

14.1 When the build fails early

1) Missing conventions in the DataCollection. **Symptom:** key errors or empty calibration basket; curve calibrates with zero instruments.

Root cause: `MARKET_DF["DATA CONVENTION"]` keys do not match the build method "INSTRUMENTS" list, or the registry file does not contain that convention.

Fix:

- print unique keys: `MARKET_DF["DATA CONVENTION"].unique()`
- verify registry contains the key: `data_convention_registry.json`
- ensure build method "INSTRUMENTS" uses convention keys (not "INSTRUMENT" labels)

2) Index key not found. **Symptom:** errors when constructing `YieldCurve` or schedule generation.

Root cause: "TARGET" index key not present in `market/index_registry.json`.

Fix: check spelling (e.g. "SOFR-1B") and ensure the index registry entry exists.

14.2 When SABR surface construction fails

3) Grid shape mismatches in SABR surfaces. **Symptom:** validation error such as `values.shape != (len(axis1), len(axis2))`.

Root cause: pivot table contains missing points or duplicated points (non-rectangular grid).

Fix:

- check missing values: `pivot.isna().sum().sum()`
- verify unique expiry/tenor pairs before pivoting
- sort indices before building `Data2D` (as in the example)

4) Wrong data_convention for SABR surfaces. **Symptom:** `SabrModel` cannot find parameters for the requested index.

Root cause: `Data2D.data_convention` must match the SABR build-method "TARGET" (index key).

Fix: ensure `data_convention=idx_name` is the index key (e.g. "SOFR-1B").

14.3 When forwards/discounting calls error

5) Overnight forward requires an end date or term. **Symptom:** `YieldCurve.forward()` throws if `termOrTerminationDate` is empty for overnight indices.

Root cause: overnight forwards require an explicit end.

Fix: pass a term like "3M" or an explicit end date.

```
fwd = yc.forward("SOFR-1B", "2026-07-01", "3M")
```

6) Non-vanilla IBOR coupon stubs. **Symptom:** error indicating end date does not match tenor-implied end date.

Root cause: `forwardIborIndex` validates that passed end dates match the index tenor end date (stub handling not enabled in this method).

Fix: call `forward()` using the standard tenor end date implied by the index tenor, or extend stub handling where required.

14.4 When risk mapping looks wrong

7) Risk vector dimension mismatch. **Symptom:** shape mismatch when solving $J^T w = g$.

Root cause: model Jacobian dimension does not match engine's risk vector dimension (often due to inconsistent calibration basket / active components).

Fix:

- confirm the model was calibrated with the same basket the Jacobian is built from
- verify `len(g)` equals the number of active calibration instruments
- recompute `jacobian()` after calibration is complete

8) Quote-space scaling confusion. **Symptom:** quote risk magnitudes appear off by $\sim 10^4$ or $\sim 10^2$.

Root cause: quote-space scaling depends on `calibration_quote_sensitivity()` and on how a quote is represented (price vs rate).

Fix:

- verify whether the node quote is a rate (decimal) or a price
- report both `param_risk` and `quote_risk` when debugging
- inspect the per-node scaling vector returned by `calibration_quote_sensitivity()`

14.5 General debugging checklist

- Print the model components and confirm the expected "TARGET" keys exist.
- Print the number of calibration instruments (basket size) and compare to Jacobian shape.
- For SABR, check each parameter surface grid is rectangular and aligned in axes.
- For a failing instrument, build it alone (single-row `MARKET_DF`) and run valuation/risk in isolation.